Roteiro de apresentação oral

RotaExpress — Kubernetes na prática · tempo alvo: 2 a 3 minutos (≈ 380 palavras)

Números para ter na ponta da língua (medianas de 3 rodadas, TAXA = 6/s)

1 Pod: 2,00 sucessos/s · 66,75% de falhas · p95 445 ms

2 Pods: 2,97 sucessos/s · 50,69% de falhas · p95 447 ms

3 Pods: 3,58 sucessos/s · 40,44% de falhas · p95 450 ms

Nenhuma atingiu a meta de falhas (< 1%). Latência e iterações descartadas (0) dentro da meta em todas.

1. O que é o projeto (≈ 20 s)

"O RotaExpress é uma aplicação de uma transportadora que calcula a menor rota entre um depósito e nove entregas. Ela funciona bem com um pedido por vez. A minha pergunta foi: **quando muitos pedidos chegam juntos, quantas cópias da aplicação são necessárias?**"

2. Docker e Kubernetes (≈ 25 s)

"Primeiro eu empacotei a aplicação com o **Docker**, na imagem `rotas:1.0`. A imagem é o pacote; o contêiner é ela rodando. Assim ela roda igual em qualquer máquina."

"Depois usei o **Kubernetes**, num cluster local com o Minikube. O Kubernetes cuida das cópias: se uma cai, ele cria outra; se eu peço mais, ele sobe mais."

3. Pod, Deployment e Service (≈ 30 s)

"O **Pod** é onde o contêiner roda. Eu não crio Pods à mão: crio um **Deployment**, que diz quantas cópias quero. Cada cópia tem meia CPU, para a capacidade ser previsível, e verificações de saúde."

"Como o IP do Pod muda toda vez que ele é recriado, criei um **Service** chamado `rotas`. Ele é um endereço fixo e distribui as conexões entre os Pods prontos."

"Também provoquei uma falha: publiquei uma versão que não existia. O Pod antigo continuou atendendo, e com `rollout undo` voltei para a versão boa."

4. Como usei o k6 (≈ 25 s)

"Para medir, usei o **k6** rodando como **Job dentro do cluster**, para passar pelo Service como um cliente real. Ele manda sempre a mesma demanda: **6 pedidos por segundo**, durante 60 segundos, depois de um aquecimento. Escolhi 6 num teste piloto: com 1 Pod houve falhas, mas também uma quantidade razoável de sucessos."

"As metas eram: **menos de 1% de falhas, p95 abaixo de 800 milissegundos e nenhum pedido descartado** pelo k6."

5. Por que 1, 2 e 3 Pods (≈ 15 s)

"Cada Pod faz **um cálculo por vez**, que leva por volta de meio segundo. Se chega outro pedido enquanto ele calcula, ele responde 503. Então, com a mesma demanda, só mudei **uma variável**: o número de Pods. Repeti cada configuração três vezes e usei a mediana."

6. Resultados (≈ 35 s)

"Com **1 Pod**: **2 sucessos por segundo** e **67% de falhas**. Ele aceita exatamente 1 de cada 3 pedidos, porque o cálculo leva uns 0,43 segundo e os pedidos chegam a cada 0,17."

"Com **2 Pods**: **quase 3 por segundo** e **51% de falhas**."

"Com **3 Pods**: **3,6 por segundo** e **40% de falhas**. Melhorou 79% em relação a 1 Pod, mas não triplicou."

"O p95 das respostas com sucesso ficou em torno de **450 ms** em todas. Ou seja: a sobrecarga não aparece como lentidão, aparece como **recusa**. Por isso olhei falhas e p95 juntos."

"Não triplicou porque o Service escolhe o Pod de cada conexão **sem saber se ele está ocupado**. Às vezes o pedido cai num Pod calculando e recebe 503, mesmo com outro livre. A carga foi bem dividida entre os Pods, mas sem olhar a ocupação."

7. Conclusão (≈ 20 s)

"Minha conclusão: **nenhuma configuração atingiu a meta de 1% de falhas**. Para essa demanda, eu usaria **3 Pods**, que reservam 1,5 CPU, porque foi a que chegou mais perto. Mas cada CPU a mais rendeu menos: de 4 sucessos por CPU com 1 Pod para 2,4 com 3."

"Aprendi que mais réplicas ajudam quando o trabalho pode ser feito em paralelo, mas não resolvem tudo: o desenho da aplicação também conta. O próximo passo seria colocar uma fila ou um balanceamento que saiba quem está livre, e medir de novo."

Frases-chave para memorizar

- Imagem é o pacote; contêiner é a execução.
- Deployment mantém as cópias; Service dá um endereço fixo.
- Readiness decide se recebe tráfego; liveness decide se reinicia.
- Mudei uma variável por vez: só o número de Pods.
- Sem fila, sobrecarga vira 503, não lentidão.
- Escolho a menor configuração que atende às metas.

Perguntas que o professor pode fazer

Pergunta	Resposta curta
Por que não criar o Pod direto?	Pod avulso não é recriado se cair. O Deployment, pelo ReplicaSet, mantém a quantidade, escala e faz rollback.
Diferença entre requests e limits?	Requests servem para o agendamento (o scheduler só coloca o Pod onde cabe). Limits são o teto: passar de CPU causa throttling; passar de memória mata o contêiner (OOMKilled).
Readiness × liveness?	Readiness falhando tira o Pod do Service sem reiniciar. Liveness falhando reinicia o contêiner.
Por que o k6 roda dentro do cluster?	Para usar o caminho real pelo Service e distribuir entre os Pods. O port-forward prende tudo em um Pod e a rede do meu computador entraria na medição.
Por que o Job aparece como Failed?	O k6 sai com código 99 quando uma meta não é atingida. O teste rodou e o resumo está no log; é resultado, não defeito.
Por que 3 Pods não zeraram as falhas?	O Service escolhe o destino de cada conexão sem saber se o Pod está ocupado. Com a taxa perto da capacidade total, um pedido ainda cai em Pod ocupado e recebe 503.
Por que olhar p95 e não só a média?	A média esconde quem esperou mais. O p95 diz que 95% das respostas com sucesso levaram até aquele tempo. E precisa ser lido junto com as falhas.
Por que o p95 não piorou com a carga?	Não há fila: quem é atendido é atendido logo; quem não é recebe 503 na hora. A sobrecarga aparece nas falhas.
Por que 1 Pod deu exatamente 2 por segundo?	Os pedidos chegam a cada 0,17 s e o cálculo leva uns 0,43 s. O Pod aceita um, recusa os dois seguintes e aceita o terceiro: 1 em 3, ou seja, 2 por segundo e 67% de falhas. Os contadores mostram +140 aceitos e −280 recusados.
Como sabe que não faltou CPU no nó com 3 Pods?	A latência média ficou igual (≈ 430 ms) em 1, 2 e 3 Pods. Se o nó estivesse sobrecarregado, cada cálculo ficaria mais lento. A perda veio da distribuição, não da CPU.

Pergunta	Resposta curta
Por que o kubectl top mostrou menos de 500m?	O metrics-server publica uma média de cerca de um minuto, que incluía tempo sem carga. É indício, não prova de saturação.
Os resultados são confiáveis?	As três rodadas de cada configuração ficaram muito próximas (1 Pod: 2,00–2,02/s; 3 Pods: 3,48–3,75/s), com ordem alternada e zero iterações descartadas. Usei a mediana.
Qual escolheria e quanto custa?	3 Pods: 3,58 sucessos/s com 1500m de CPU reservada. Não atinge a meta de falhas; para isso eu mudaria a aplicação (fila ou balanceamento por ocupação) antes de só somar réplicas.
O que aconteceu com a imagem rotas:1.1?	Ela não existe. O Pod novo ficou em ErrImagePull/ImagePullBackOff, nunca ficou pronto, então nunca recebeu tráfego. O antigo continuou atendendo e o rollout undo voltou à revisão boa.
Como escalaria automaticamente?	Com HorizontalPodAutoscaler, usando o metrics-server e os requests de CPU. Ex.: kubectl autoscale deployment rotas --cpu-percent=70 --min=1 --max=3. Mas só depois de medir a capacidade de uma réplica, como fiz aqui.
O que é drift?	Se escalo com kubectl scale e depois alguém aplica o deployment.yaml com replicas: 1, volta para 1. O arquivo é a fonte da verdade; o certo é mudar o arquivo (de preferência por pipeline/GitOps).
Por que pedir 6 réplicas deixou Pods Pending?	O nó tem 3 CPUs e cada Pod reserva 500m. O scheduler soma os requests; quando não cabe, o Pod fica Pending com Insufficient cpu, mesmo com CPU ociosa.
Por que não usar a tag latest?	latest não diz qual versão está rodando, dificulta rollback e por padrão força novo download. Uso tag por versão (1.0, 1.1).
Namespace isola aplicações?	É uma divisão de nomes e organização, não uma barreira de segurança. Isolamento vem de RBAC, NetworkPolicy, ResourceQuota etc.
Três Pods garantem alta disponibilidade?	Não neste laboratório: os três estão no mesmo nó. Se o nó cair, caem todos. Em produção, eu distribuiria entre nós e zonas.